

UNIVERSIDAD DISTRITAL FRANCISCO JOSE DE CALDAS
FACULTAD DE INGENIERÍA
PROYECTO CURRICULAR DE INGENIERÍA DE SISTEMAS
CIENCIAS DE LA COMPUTACIÓN 3 (Grupo 020-81)

Profesor: Helio Henry Ramírez Arévalo

Estudiantes:

Janeth Oliveros Ramírez
20182020100

Hemerson Julian Ballen Triana
20211020084



Manual Técnico sobre los ejercicios de pila y cola

1. Descripción general

Aplicación web que resuelve dos ejercicios de estructuras de datos en una sola interfaz con pestañas: el equilibrado de símbolos (signos de apertura y clausura del teclado) mediante un tipo de dato abstracto Pila, y la asignación de tareas mediante un tipo de dato abstracto Cola. La aplicación se ejecuta por completo en el navegador. El cual se puede visualizar en el siguiente enlace:

<https://pila-cola-ejercicios-propuestos-peach.vercel.app/>

Y cuyos archivos de código se pueden encontrar en el siguiente repositorio:

<https://github.com/JOliverosRIng/Pila-Cola-Ejercicios-Propuestos.git>

2. Requisitos y ejecución

Solo requiere un navegador (Chrome, Firefox o Edge), ya sea desde el enlace de visualización o desde el repositorio. Para abrir el programa desde el repositorio se hace la descarga de la carpeta a su computador y se hace doble clic sobre index.html. No necesita conexión a internet ni dependencias externas.

3. Tecnologías utilizadas

La aplicación se construyó con las tres tecnologías nativas de la web, sin frameworks ni librerías externas:

<i>Tecnología</i>	<i>Rol en el proyecto</i>	<i>¿Por qué se eligió?</i>
<i>HTML</i>	Estructura de la página: pestañas, formularios, tablas y contenedores.	Es el estándar para estructurar contenido web; corre nativo en todo navegador.
<i>CSS</i>	Presentación: estilos, cajas de la pila y línea de tiempo de la cola.	Permite un diseño claro con reglas propias, sin cargar librerías pesadas.
<i>JavaScript</i>	Lógica: clases de los distitos tipos de datos abstractos, algoritmos e interactividad.	Se ejecuta en el navegador sin compilar; permite implementar los TDA facilmente.

Se eligió deliberadamente JavaScript sin frameworks de apoyo como React o Vue, ni librerías como jQuery o Bootstrap, por tres razones:

- ❖ Simplicidad de entrega: Un solo conjunto de archivos que se abre con doble clic, sin paso de compilación ni instalación.

- ❖ Valor aprendido: El ejercicio exige demostrar los TDA. Usar una librería con estructuras ya hechas ocultaría justamente lo que se evalúa; por eso la pila y la cola se programaron a mano.
- ❖ Peso y mantenimiento: Para dos módulos pequeños, un framework añadiría complejidad sin beneficio. Las tres tecnologías base cubren todas las necesidades del proyecto.

4. Arquitectura y diagramas

El código se organiza en dos módulos independientes, uno por cada estructura de datos, más el control de las pestañas. Cada módulo contiene su clase TDA, su algoritmo y su interfaz. La lógica se mantiene separada de la presentación: las clases no manipulan la pantalla directamente, lo que facilita probarlas y repartir el trabajo.

```
Pila-Cola-Ejercicios-Propuestos/
|—— index.html          Estructura y pestañas
|—— css/
|   |—— estilos.css      Presentación
|—— js/
|   |—— pila.js           Módulo Pila
|   |—— cola.js          Módulo Cola
|   |—— app.js           Pestañas y arranque
```

Vista de los archivos. Arquitectura.

Diagrama de casos de uso:

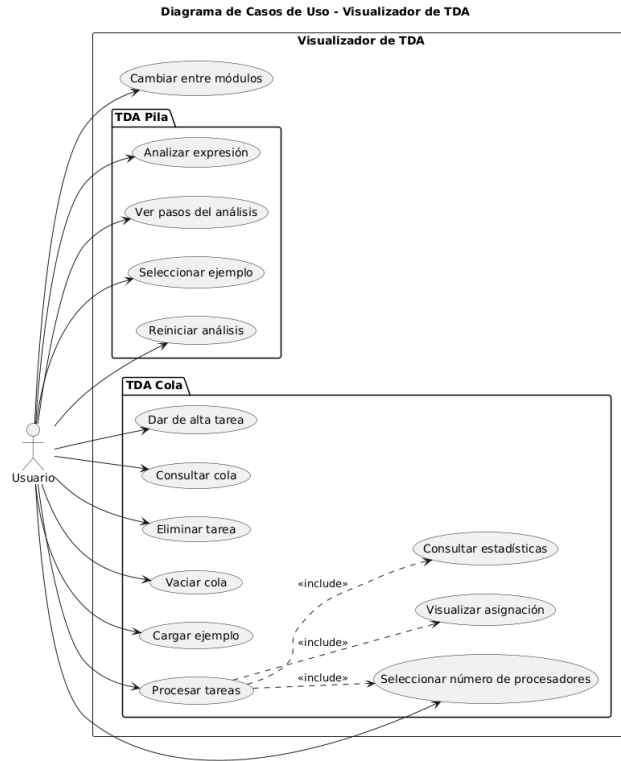
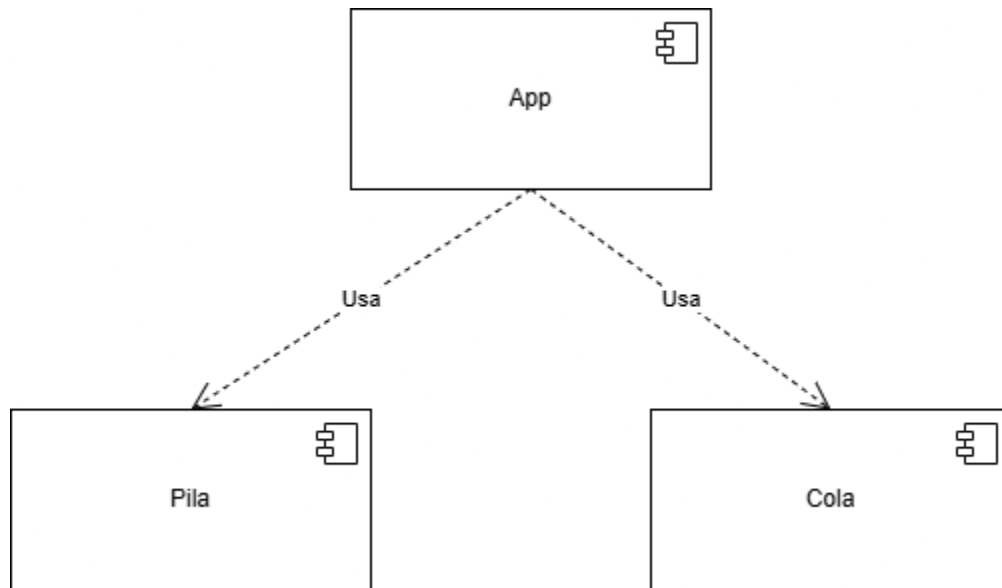


Diagrama de componentes:



5. Módulo Pila

Equilibrado de símbolos

Tipo de dato abstracto Pila (LIFO)

Esta clase contiene las operaciones *push*, *pop*, *peek* y *estaVacía*. Los elementos se guardan en un arreglo privado que no se expone al exterior, respetando la abstracción del tipo de dato.

Algoritmo

El algoritmo recorre la expresión ingresada por el usuario carácter por carácter y dependiendo de una lista de caracteres como lo son: “(”, “[“, “{“, que llamaremos de apertura; se ingresaran en la pila por orden de llegada. Ante un símbolo de cierre como lo son “)”, “]”, “}” lo compara con la cima, entonces, si la pila está vacía da error; si la cima es la apertura correspondiente al símbolo, la extrae y saca de la cola; si no coincide, es error. Al terminar, si la pila no quedó vacía, da error. Para lo demás cualquier otro carácter se ignora.

Ejemplo con la expresión {h[o(l)a]}

El primer carácter detectado corresponde a la lista de caracteres de apertura, por lo tanto, ingresa a la pila en la posición 0, el segundo carácter se ignora pues no es ni de apertura ni de cierre, el tercero entra a la pila y queda en la posición 1, el siguiente carácter se ignora y el que le sigue entra a la pila en la posición 2, continuamos con otro carácter que se ignora y llegamos al primer carácter de cierre; este debe compararse con el ultimo elemento de la pila que en este caso es el elemento en la posición 2, se analiza si el carácter de esa posición corresponde al carácter de apertura adecuado y cómo se cumple con la regla el último elemento en la pila se libera y se pasa al siguiente carácter de la expresión. Para esta expresión el algoritmo recorrerá los caracteres y encontrará las respectivas parejas sin error alguno.

Uso

Se escribe la expresión en el textbox y con el botón *Analizar*, se pasa por el algoritmo y se obtiene el resultado; con el botón *Paso a paso*, se anima la pila creciendo y decreciendo dando el tiempo suficiente para apreciar el cambio en la estructura de datos, así podemos evidenciar el comportamiento del algoritmo. Asimismo, el programa indica el tipo de error y la posición donde ocurre. Por último, el botón *Reiniciar* limpia el Textbox y el elemento visual que representa la pila, para que el usuario pueda escribir una nueva expresión y analizarla.

6. Módulo Cola

Asignación de tareas

Tipo de dato abstracto Cola (FIFO)

Esta clase contiene las operaciones *encolar*, *desencolar*, *frente* y *estaVacía*, con un arreglo interno privado.

Operaciones de la aplicación

Al insertar una tarea nueva se solicita un identificador y su duración. La eliminación de cada tarea la acompaña, así que se va a representar en una cola auxiliar por medio de una tabla la llegada de cada tarea, de esta manera se cumple también con la disciplina FIFO, mostrando la cola del frente al final, y procesando todas las tareas.

Planificación en m procesadores

De acuerdo con el planteamiento del ejercicio el usuario tendrá la oportunidad de indicar el número de procesadores *m*. Las tareas se ordenan por duración de menor a mayor y se asignan al procesador que

quede libre antes. El resultado se muestra como una línea de tiempo por procesador, junto con el tiempo de finalización de cada tarea, la suma total de los tiempos de las tareas, el tiempo promedio y el makespan (en cuánto tiempo terminó el procesamiento de todas las tareas).

7. Por qué SPT en lugar de FIFO

El enunciado pide minimizar el tiempo medio de finalización. Una cola FIFO pura atiende las tareas en su orden de llegada, lo cual es justo pero no minimiza ese promedio. Por eso se aplicó la regla SPT (Shortest Processing Time, la tarea más corta primero).

La razón es que, al ejecutar en secuencia, cada tarea retrasa a todas las que vienen después. Si una tarea larga va primero, su duración se suma al tiempo de espera de todas las demás. Colocando las cortas primero, ese retraso acumulado se reduce al mínimo. Es un resultado demostrable: SPT produce el menor tiempo medio de finalización posible.

Ejemplo con las tareas A=5, B=2, C=8, D=1 en un procesador:

<i>Orden</i>	<i>Secuencia de tareas</i>	<i>Suma de finalizaciones</i>	<i>Tiempo medio</i>
FIFO (llegada)	A, B, C, D	$5 + 7 + 15 + 16 = 43$	10.75
SPT (más corta primero)	D, B, A, C	$1 + 3 + 8 + 16 = 28$	7.00

Con los mismos datos, SPT reduce el tiempo medio de 10.75 a 7.00: una mejora del 35 % sin cambiar el trabajo total del procesador (el makespan sigue siendo 16).

8. Validaciones y limitaciones

- ❖ La duración de cada tarea debe ser un entero de 1 o más.
- ❖ No se admiten identificadores de tarea repetidos.
- ❖ Si se indican más procesadores que tareas, el exceso no aporta mejora.

9. Conclusiones y aprendizajes

Podemos concluir que para implementar los TDA con encapsulamiento (datos privados, acceso solo por las operaciones) se permite usarlos sin conocer su interior, y cambiar la implementación sin afectar el resto del programa. Esto confirma el valor de programar con tipos abstractos y no con arreglos sueltos. Además, para la cola el orden en que se ejecutan las tareas sí afecta el tiempo medio de finalización, aunque el trabajo total del procesador sea el mismo. Con nuestros datos, pasar de FIFO a SPT redujo el promedio de 10.75 a 7.00 sin cambiar el makespan. La regla SPT (la tarea más corta primero) minimiza el tiempo medio de finalización, mientras que FIFO, aunque más simple y justo por orden de llegada, no lo optimiza. Esto muestra que elegir bien la política de planificación importa tanto como la estructura de datos.

Más procesadores reducen el tiempo medio, pero con rendimientos decrecientes hasta llegar al piso, cuando hay tantos procesadores como tareas, el problema se vuelve trivial. Separar el proyecto en dos módulos con lógica desacoplada de la interfaz permitió el trabajo colaborativo. El enunciado de las colas presenta un ejercicio que pide "minimizar el tiempo medio" pero menciona " n procesadores para n tareas", caso en que no hay nada que optimizar. El análisis reveló que el problema es significativo solo cuando hay menos procesadores que tareas, y que reconocer los límites del propio enunciado es parte de entenderlo.